Log Based Recovery

Question 1:
<START T1>
<T1, A, 100, 150>
<T1, B, 200, 250>
<START T2>
<T2, C, 300, 350>
--- SYSTEM CRASH —

1. Identify which transactions need to be undone and which need to be redone.
2. Show the final values of A, B, and C after recovery.
3. Explain why UNDO or REDO is required in this case.

Solution:
1. T1 Transaction and T2 transaction both needs to be undone as none of the transactions were committed. No transactions need to be redone.
2. Final Values after recovery, A = 100, B = 200 and C = 300
3. Since no transaction was committed, none of the updates are durable. Log based protocol ensures old values are available, so we rollback everything. This is a pure UNDO recovery case.

Question 2:
<START T1>
<T1, A, 500, 600>
<COMMIT T1>
<START T2>
<T2, B, 1000, 1200>
<START T3>
<T3, C, 700, 800>
<COMMIT T3>
--- SYSTEM CRASH —

1. Identify which transactions need to be undone and which need to be redone.
2. Show the final values of A, B, and C after recovery.

Solution:
1. T1 Transaction and T3 transaction both needs to be redone as the transactions were committed. On the other hand, T2 transactions need to be undo as it was not committed.
2. Final Values after recovery, A = 600, B = 1000 and C = 800

Question 3:
A banking database tracks customer transactions. Two transactions run concurrently:

Transaction T1 (Teller): Processes a withdrawal for Customer X.
Transaction T2 (Auditor): Generates a report on all customers with a balance below $100.

- T1 reads Customer X's balance as $150.
- T2 starts generating a report and reads Customer X's balance as $150.
- T1 deducts $100 from Customer X's balance, setting it to $50 (but has **not committed yet**).
- T2 reads Customer X's balance again and sees the **uncommitted** $50 value.
- T1 **rollbacks** the transaction, reverting Customer X's balance to $150.
- Meanwhile, T2 inserts a new customer, Customer Y, with a balance of $80.

- T2 re-runs the query for customers with a balance below $100 and now finds Customer Y (but not Customer X).

Questions:

1. Identify the concurrency problems in the scenario. Provide your reason.
2. State which ACID property is being violated for each problem.
3. How can these issues be prevented?
4. What type of log based recovery needs to be used to revert back to the original balance?

Solution:

1. Concurrency Control Problems:
   Dirty Read: T2 reads Customer X's uncommitted balance of $50.
   Phantom Read: T2's second query finds Customer Y (a new row) that was not present in the first query.
2. Isolation violated.
3. Use SERIALIZABLE isolation level to execute the transactions one after another.
4. Undo log keeps track of the old values, so it is used to go back to the original values.